

Nutzung von Zustandsdiagrammen zur Modellierung von Benutzeroberflächenelementen - eine Fallstudie

Studienarbeit

Hoentzsch, Christoph
Softwareprojekt
Matrikelnummer: 3005551

Gutachter: Prof. Dr. Steffen Greiffenberg

Tag der Einreichung: 17.12.2025

KI-Vermerk

Die vorliegende wissenschaftliche Arbeit wurde mit Unterstützung einer generativen KI erstellt. Inhalte wurden teilweise auf Basis der Ausgabe der Künstlichen Intelligenz erstellt, wobei eine eigenständige Überprüfung und wissenschaftliche Validierung durch die Autorin bzw. den Autor erfolgte. Die Verantwortung für die Richtigkeit, Vollständigkeit und wissenschaftliche Relevanz der Arbeit liegt bei der Autorin bzw. dem Autor. Die Nutzung der KI erfolgte ausschließlich zur Unterstützung der Erstellung und Analyse von Textinhalten im Rahmen der wissenschaftlichen Arbeit. Die benutzten Modelle sind GPT 4.1 und GPT 5.

Inhaltsverzeichnis

Inhaltsverzeichnis	III
Abbildungsverzeichnis	IV
Abkürzungsverzeichnis	V
1 Einleitung	1
2 Theoretischer Hintergrund	2
2.1 Unified Modeling Language	2
2.2 Zustandsdiagramme	4
2.2.1 Definition	4
2.2.2 Anwendung	6
2.3 Grafische Benutzeroberflächen	8
2.3.1 Definition	8
2.3.2 Entwicklungsprozess	8
2.4 Zustandsdiagramme in der Entwicklung von grafischen Benutzeroberflächen	10
3 Methodik	11
3.1 Forschungsmethode	11
3.2 Vorgehen	11
4 Empirische Beobachtung	13
4.1 Fallbeschreibung	13
4.1.1 Das Gartenbahnprojekt	13
4.1.2 Der Controlleur	14
4.1.3 Redesign der Benutzeroberfläche	15
5 Analyse	21
6 Verifizierung und Validierung	23
6.1 Verifizierung als wissenschaftliches Schreibprojekt	23
6.2 Validierung	24
6.2.1 Wissenschaftliche Relevanz	24
6.2.2 Relevanz der Arbeit für den Autor	24
7 Fazit und Ausblick	26
Literatur	27

Abbildungsverzeichnis

Abbildung 1:	Diagrammarten der UML	3
Abbildung 2:	Elemente von Zustandsdiagrammen	5
Abbildung 3:	Definition Finiter Zustandsautomat	6
Abbildung 4:	Definition Mealy-Automat	6
Abbildung 5:	Aufgaben eines Statecharts	7
Abbildung 6:	Benutzeroberfläche des Clients. Die obere Tableiste stellt die Liste an verfügbaren Modellen dar	14
Abbildung 7:	Mockup der Benutzeroberfläche	16
Abbildung 8:	Zustandsdiagramm AttributeCard	16
Abbildung 9:	Zustandsdiagramm ButtonPressedIndicator	17
Abbildung 10:	Zustandsdiagramm SpeedDial	17
Abbildung 11:	Die entstandene Benutzeroberfläche	19
Abbildung 12:	Die Benutzeroberfläche ändert sich durch das Drücken eines Knopfes	19
Abbildung 13:	Die Benutzeroberfläche ändert sich, weil die Geschwindigkeit der Lokomotive geändert und ein Attribut der Lokomotive auf true gesetzt wurde	20

Abkürzungsverzeichnis

GUI Grafische Benutzeroberfläche

SCXML Statechart-XML

UML Unified Modeling Language

1 Einleitung

Damit Menschen mit Software interagieren können, benötigt es einer Schnittstelle. Um Daten und Informationen an Menschen auszugeben, wird in zeitnaher Software oft eine grafische Benutzeroberfläche (graphical user interface, GUI) verwendet. Grafische Benutzeroberflächen beanspruchen in der Softwareentwicklung viel Aufmerksamkeit, da sie die Nutzbarkeit von Software stark beeinflussen können. Um die Konzeption, Gestaltung und Entwicklung von Benutzeroberflächen zu erleichtern, werden verschiedene Hilfsmittel verwendet.

Die Unified Modeling Language (UML) ist eine Sprache, mit der Diagramme beschrieben werden können. Diese Diagramme können zur Modellierung in der Softwareentwicklung genutzt werden. Zustandsdiagramme, StateMachines oder Statecharts können zur Darstellung von endlichen Automaten genutzt werden. Sie beschreiben deren Verhalten durch eine Menge an Zuständen und Zustandsübergängen.

In dieser Arbeit soll die Anwendung von UML Zustandsdiagrammen in der Entwicklung von grafischen Benutzeroberflächen untersucht werden. Dies erfolgt in Form einer Fallstudie. Der Anwendungsfall basiert auf der Entwicklung einer Software zur Steuerung einer Gartenbahn mit einem Videospiel-Controller, welche im Rahmen des Moduls „Softwareprojekt“ an der Dualen Hochschule Sachsen in Dresden durchgeführt wurde.

Das Ziel dieser Arbeit ist es, die folgende Forschungsfrage zu beantworten:

Wie können UML-Zustandsdiagramme in der Entwicklung von Benutzeroberflächen genutzt werden?

Um diese Frage zu beantworten, werden im theoretischen Teil dieser Arbeit die Grundlagen zu UML-Zustandsdiagrammen, grafischen Benutzeroberflächen und deren Entwicklung erarbeitet. Dabei sollen folgende untergeordnete Forschungsfragen beantwortet werden:

- Wie sind Zustandsdiagramme im Rahmen der UML definiert?
- Wie werden Zustandsdiagramme angewendet?
- Wie werden Benutzeroberflächen entwickelt?

2 Theoretischer Hintergrund

2.1 Unified Modeling Language

Die Unified Modeling Language (UML) ist eine allgemein verwendbare Modellierungssprache.¹ Unter Anwendung des semiotischen Dreiecks sind die Diagramme und Diagrammelemente die Zeichen, das Modell der Begriff und der modellierte Gegenstand das Objekt. Ein Modell stellt eine Abstraktion des modellierten Gegenstandes dar, wobei bei UML Modellen zusätzlich die Perspektive und das Ziel des Modells betrachtet werden.²³ Als Zweck der Modellierung wird in einigen Begriffsdefinitionen die Erleichterung einer Untersuchung gegeben.⁴ Der modellierte Gegenstand sind in der UML im allgemeinen Systeme, welche durch Klassifikatoren bezeichnete Objekte, Ereignisse und Verhalten enthalten.⁵ Die UML kann in verschiedensten Anwendungsgebieten angewendet werden.⁶ Der Hauptanwendungsbereich ist die Softwareentwicklung.⁷⁸

Die UML unterscheidet die UML Diagrammelemente in verschiedene zwei verschiedene semantische Gebiete, die sich über die zeitliche Betrachtung unterscheiden. Strukturelemente für Objekte werden durch die Struktursemantik definiert und können zu einem bestimmten Zeitpunkt wahr sein. Verhaltenselemente werden durch die Verhaltenssemantik definiert und beschreiben Objekte über einen bestimmten Zeitraum.⁹ Daraus entstehen drei Modellierungsarten:¹⁰¹¹

- Strukturmodellierung (enthält Elemente für Klassifikatoren, Pakete, Werte, Klassen, etc.)
- Verhaltensmodellierung (enthält Elemente für Zustandsmaschinen, Aktivitäten, Interaktionen, Aktionen, etc.)
- Ergänzungsmodellierung (enthält Elemente für Anwendungsfälle, Deployments, Informationsflüsse)

Die Abbildung 1 stellt eine Übersicht über die Diagrammart, die mit der UML assoziiert werden, da.

¹ Kecher, C./Hoffmann-Elbern, R./Will, T. T. (2021), S. 22.

² Vgl. Group, O. M. (2017), S. 12.

³ Vgl. Rumpe, B. (2011), S. 9.

⁴ Vgl. ebenda, S. 9.

⁵ Vgl. Group, O. M. (2017), S. 12.

⁶ Vgl. Kecher, C./Hoffmann-Elbern, R./Will, T. T. (2021), S. 22.

⁷ Vgl. ebenda, S. 22.

⁸ Vgl. Group, O. M. (2017), S. 1.

⁹ Vgl. ebenda, S. 13.

¹⁰ Vgl. ebenda, S. 14.

¹¹ Vgl. Kecher, C./Hoffmann-Elbern, R./Will, T. T. (2021), S. 26.

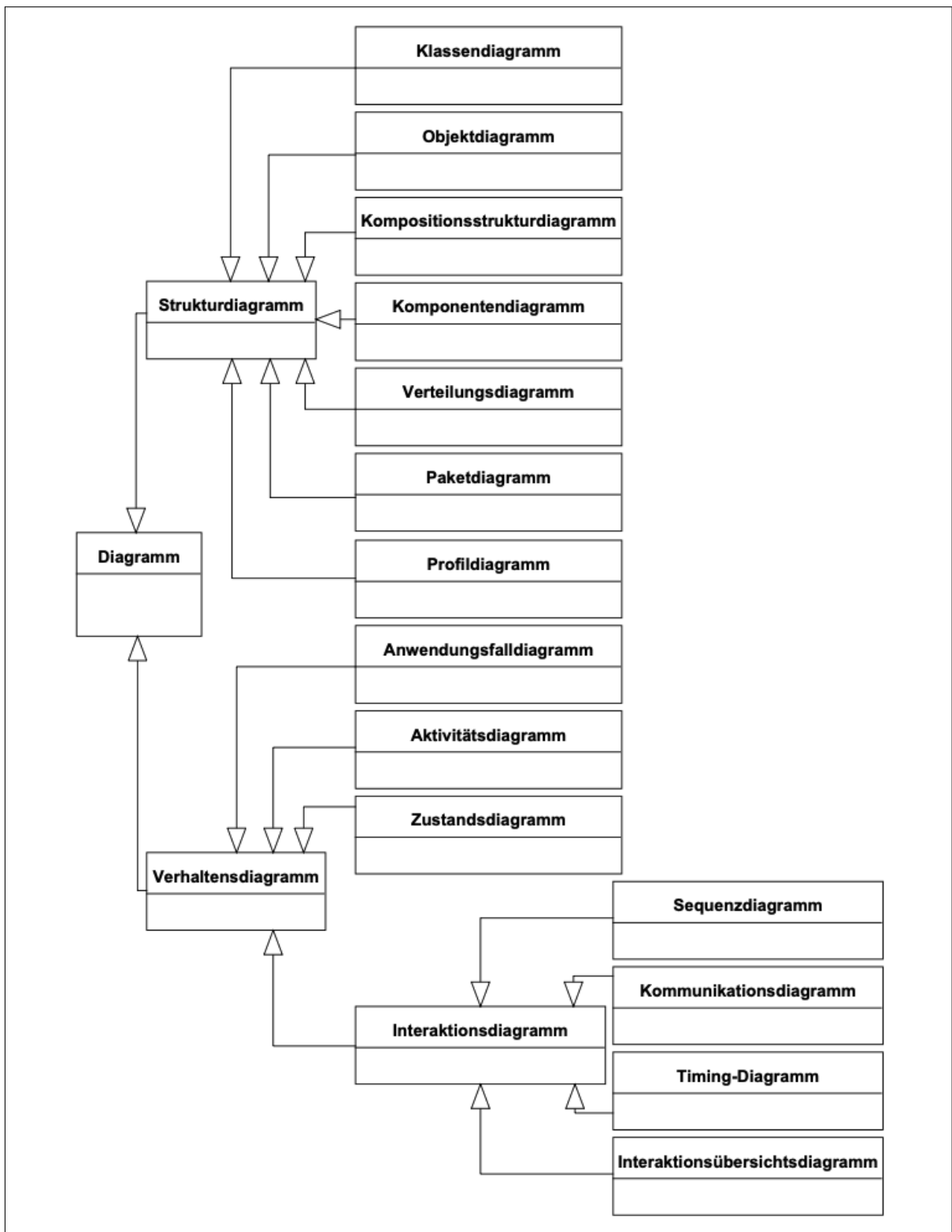


Abbildung 1: Diagrammarten der UML (Quelle: Kecher, C./Hoffmann-Elbern, R./Will, T. T. (2021))

2.2 Zustandsdiagramme

2.2.1 Definition

Zustandsdiagramme sind Teil der Verhaltensmodellierung. Sie basieren auf dem „Statechart“-Formalismus von David Harel.¹²¹³ Zustandsdiagramme modellieren das diskrete ereignisbasierte Verhalten des modellierten Gegenstandes in Form von endlichen Zustandsautomaten.¹⁴ Es wird in zwei verschiedene Arten von Zustandsdiagrammen unterschieden: „Verhaltenszustandsdiagramme“ (behavior state machines) stellen das Verhalten eines Objekts dar, während „Protokollzustandsdiagramme“ (protocol state machines) die möglichen Interaktionssequenzen für Objekte darstellen.¹⁵ In dieser Arbeit werden nur Verhaltenszustandsdiagramme betrachtet, da diese im praktischen Fall angewendet wurden. Die folgende Beschreibung von (Verhaltens-) Zustandsautomaten wurde aus der UML Spezifikation Sektion 14.2.3 und aus Rumpe 2011 Kapitel 5.3f entnommen.¹⁶¹⁷

Ein Zustandsautomat ist zusammengesetzt aus Regionen, die einen Graph aus Eckpunkten und Kanten enthalten. Die Eckpunkte des Graphen können Zustände, Pseudozustände oder Verbindungspunkte darstellen. Die Kanten des Graphen stellen Transitionen dar.

Ein Zustand ist eine Situation während der Ausführung des Zustandsautomaten, in der eine bestimmte Bedingung gilt. Diese Bedingung wird meist durch den Namen des Zustands impliziert. Ein Zustand kann verschiedene interne Verhalten/Aktionen besitzen, die bei dem Betreten (entry), nach dem Betreten für die Dauer des Zustands (do) und nach dem Verlassen des Zustands (exit) ausgeführt werden. Es existieren drei Arten von Zuständen:

- Simple Zustände enthalten keine Graphen
- Zusammengesetzte Zustände enthält mindestens eine Region
- Unterzustandsautomatenzustände enthalten einen Zustandsautomat

Zusammengesetzte Zustände können zudem eine Historie besitzen, welche die Zustandskonfiguration vor dem letzten Verlassen des Zustands darstellen. Die Tiefhistorie und die Flachhistorie unterscheiden sich in der Tiefe der „gemerkten“ Zustände; die Tiefhistorie merkt sich die vollständige Zustandskonfiguration, während sich die Flachhistorie nur

¹² Vgl. Group, O. M. (2017), S. 305.

¹³ Vgl. Harel, D. (1987).

¹⁴ Vgl. Group, O. M. (2017), S. 305.

¹⁵ Vgl. ebenda, S. 305.

¹⁶ Vgl. ebenda, S. 306–318.

¹⁷ Vgl. Rumpe, B. (2011), 150ff.

die oberste Teilzustandsebene merkt. Der sog. Endzustand zeigt an, dass die Ausführung einer Region vollendet ist.

Pseudozustände sind sehr kurzweilige Situationen, die während der Ausführung des Zustandsautomaten angenommen werden können. Relevant ist für diese Arbeit der Startzustand, der den Startpunkt für eine Region darstellt und von dem die Region bei Standardeintritt ausgeführt wird. Verbindungspunkte können Ein- und Austrittspunkte für Unterzustandsautomatenzustände darstellen.

Transitionen sind gerichtete Übergänge zwischen einem Quelleckpunkt und einem Zieleckpunkt und stellen einen Teil des Verhaltens des Zustandsautomaten dar. Eine Transition wird ausgeführt, wenn ein Event für einen Stimulus (trigger) der Transition eintritt und wenn sich der Zustandsautomat an der Quellkante befindet.

Zustandsdiagramme stellen Zustandsautomaten grafisch dar. Die voran beschriebenen Elemente sind in Abbildung 2 in der UML Notation dargestellt.

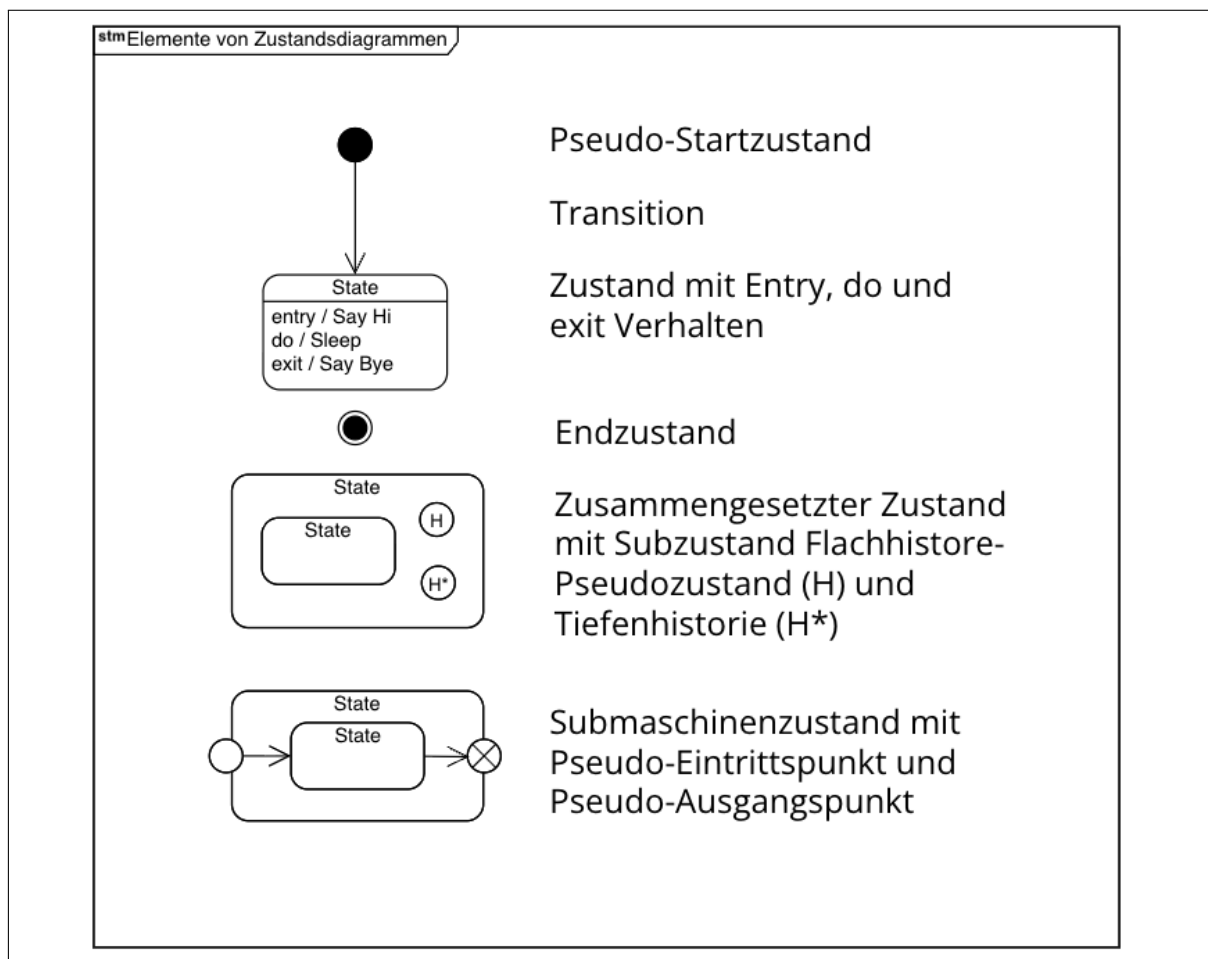


Abbildung 2: Elemente von Zustandsdiagrammen (Quelle: Eigene Darstellung)

2.2.2 Anwendung

Die Zustandsautomaten und die vorausgehenden Statecharts basieren auf der finiten Automatentheorie. Es kann dabei in zwei Typen von Automaten unterschieden werden:¹⁸

- Erkennende Automaten durchlaufen für ein Eingabealphabet eine Menge an Zuständen durch Transitionen von einem Start- zu einem Endzustand. Eine Definition ist in Abbildung 3 zu finden.
- Mealy-Automaten geben zusätzlich an jeder Transition ein Element eines Ausgabealphabets aus. Eine Definition ist in Abbildung 4 zu finden.

Ein *erkennender Automat* (auch *nichtdeterministischer, alphabetischer Rabin-Scott Automat (RSA)*) besteht aus einem Fünftupel (Z, E, t, S, F) mit den folgenden Komponenten:

- einer Menge von Zuständen Z ,
- einem Alphabet E , auch Eingabealphabet genannt,
- einer Menge von Startzuständen $S \subseteq Z$,
- einer Menge von Endzuständen $F \subseteq Z$ und
- einer Transitionsrelation $t \subseteq Z \times E^\varepsilon \times Z$.

Dabei ist E^ε das Eingabealphabet E erweitert um das Leerzeichen ε , das für spontane Transitionen verwendet wird. Normalerweise werden die beteiligten Mengen als endlich angenommen. Alle Mengen seien nicht leer.

Eine Transition ist *schaltbereit*, wenn im Quellzustand das entsprechende Zeichen anliegt oder die Transition kein Eingabezeichen verlangt (also spontan ist).

Abbildung 3: Definition Finiten Zustandsautomat (Quelle: Rumpe, B. (2011))

Ein (alphabetischer, nichtdeterministischer und unvollständiger) *Mealy-Automat* ist ein Sechstupel (Z, E, A, t, S, F) , das einen erkennenden Automaten (Z, E, t, S, F) beinhaltet und zusätzlich

- ein Ausgabealphabet A und
- eine um eine Ausgabe erweiterte Transitionsrelation $t \subseteq Z \times E^\varepsilon \times A \times Z$ hat.

Abbildung 4: Definition Mealy-Automat (Quelle: Rumpe, B. (2011))

Aufgrund des hohen Grades an Abstraktion der Automatentheorie kann sie auf zahlreiche Gebiete angewendet werden. Die von Rumpe formulierten Aufgaben Aufgaben, die ein Zustandsdiagramm in der Objektorientierten Softwareentwicklung übernehmen kann, werden in Abbildung 5 dargestellt:¹⁹

¹⁸ Rumpe, B. (2011), S. 151–152.

¹⁹ ebenda, S. 149.

Die (überlappenden) Aufgaben eines Statecharts können sein:

- Darstellung des Lebenszyklus eines Objekts
- Implementierungsbeschreibung einer Methode
- Implementierungsbeschreibung des Verhaltens eines Objekts
- Anforderungen an den (abstrakten) Zustandsraum eines Objekts
- Darstellung der Reihenfolge von erlaubten Eintreten von Stimuli (Aufrufreihenfolge)
- Charakterisierung der möglichen oder erlaubten Verhalten eines Objekts
- Bindeglied zwischen Zustands- und Verhaltensbeschreibung

Abbildung 5: Aufgaben eines Statecharts (Quelle: Rumpe, B. (2011))

In konkreten Anwendungsszenarien können Zustandsdiagramme unter anderem für Softwaretesting und die Erstellung von Testfällen²⁰²¹, zur Automation in der Fertigung²² oder für Automatische Code Generierung²³ genutzt werden.

²⁰ Vgl. Aggarwal, M./Sabharwal, S. (2012).

²¹ Vgl. Felderer, M./Herrmann, A. (2014).

²² Vgl. Guo, W. (/Zhou, M./Wei, D. (2007).

²³ Vgl. V., S. E./Samuel, P. (2019).

2.3 Grafische Benutzeroberflächen

2.3.1 Definition

Eine grafische Benutzeroberfläche (auch Benutzeroberfläche, GUI oder UI) ist eine Schnittstelle zwischen Menschen und Maschinen.²⁴²⁵²⁶²⁷ Der Mensch benutzt die Benutzeroberfläche, um der Maschine Anweisungen zu geben und Informationen aufzunehmen.²⁸²⁹ Benutzeroberflächen sind meist Ereignis-basiert; eine Aufgabe wird im Programm ausgeführt, wenn der Nutzer eine Aktion ausführt, beispielsweise das „klicken“ eines Elements der grafischen Benutzeroberfläche.³⁰ Die Benutzeroberfläche kann dabei effektiv einschränken, welche Aufgaben ausgeführt werden können.³¹ Grafische Benutzeroberflächen enthalten und nutzen in ihrer Darstellung meist Fenster, Symbole, Menüs, Knöpfe, Dropdowns, Eingabefelder und Zeiger.³²³³ Grafische Benutzeroberflächen werden oft mit der Befehlszeile in Kontrast gesetzt, welche dem Nutzer keinen Kontext geben und Vorkenntnisse über das laufende Programm voraussetzen.³⁴³⁵

2.3.2 Entwicklungsprozess

Der Entwicklungsprozess von grafischen Benutzeroberflächen und der allgemeine Entwicklungsprozess sind stark miteinander verknüpft.³⁶ Allgemein gilt, dass die Phasen, die allgemein bei der Softwareentwicklung durchlaufen werden, ebenfalls bei der Entwicklung von grafischen Benutzeroberflächen auftreten, da letzteres Teil des ersteren ist. Modelle wie das Wasserfallmodell oder das Inkrementelle Entwicklungsmodell mit Methoden wie Scrum oder Test-Driven Development können auch den Rahmen für die Entwicklung von grafischen Benutzeroberflächen bilden.³⁷

Besonders anzumerken ist, dass die Nutzerzentrierung bei der Benutzeroberflächenentwicklung stark betont wird, weil Endnutzer auf die Nutzbarkeit der Benutzeroberfläche

²⁴ Vgl. Chlebek, P. (2011), S. 4.

²⁵ Vgl. Jansen, B. J. (1998), S. 1.

²⁶ Vgl. Martinez, W. L. (2011), S. 1.

²⁷ Vgl. Heinzl, A./Mädche, A./Riedl, R. (2024), S. 150.

²⁸ Vgl. Jansen, B. J. (1998), S. 1.

²⁹ Vgl. Martinez, W. L. (2011), S. 1.

³⁰ Vgl. ebenda, S. 1.

³¹ Vgl. Jansen, B. J. (1998), S. 1.

³² Vgl. Martinez, W. L. (2011), S. 1.

³³ Vgl. Jansen, B. J. (1998), S. 1.

³⁴ Vgl. Martinez, W. L. (2011), S. 1.

³⁵ Vgl. Jansen, B. J. (1998), S. 1.

³⁶ Vgl. V.N. Lukin A. L. Dzyubenko, Y. B. C. (2019), S. 318.

³⁷ Vgl. Sommerville, I. (2016), S. 47–50.

angewiesen sind.³⁸³⁹ In der Analyse und im Entwurf sind Techniken wie Anwendermodelle, Anforderungsinterviews oder Storyboards explizit auf die nutzerzentrierte Anforderungserhebung ausgelegt.⁴⁰ Die Nutzung von Dialogseitendiagrammen und Screendesigns kann Entwicklern in der Entwurfsphase helfen bei der Veranschaulichung und können iterativ eingesetzt werden, um Nutzerfeedback zu sammeln.⁴¹ Die Erstellung von Prototypen in der Benutzeroberflächenentwicklung dient nicht nur der Verifizierung, sondern mithilfe von Nutzertests auch der Validierung.⁴²⁴³ Außerdem werden in der Entwicklung von Benutzeroberflächen dem (initialen) Design und dem Redesign (Verbesserung von etwas Bestehendem aufgrund von Nutzerfeedback) ähnliche Priorität gegeben.⁴⁴

Für die Programmierung von grafischen Benutzeroberflächen existieren je nach Plattform eine hohe Anzahl an verschiedenen Werkzeugen, Bibliotheken und Frameworks. Für die Android Plattform ist Jetpack Compose eine offiziell (von Google, Inhaber von Android) unterstützte UI-Bibliothek. Jetpack Compose baut deklarativ grafische Benutzeroberflächen. Elemente der Benutzeroberfläche werden als im Code als Funktionen dargestellt, welche wiederverwendet und ineinander verschachtelt werden können. Daten aus der Geschäftslogik können dabei als Parameter an die Elemente weitergegeben werden. Die lokale Compose-Laufzeit übernimmt die Komposition (initialer Aufbau der Benutzeroberfläche) und Rekombination (Aktualisierung der Elemente basierend auf Ereignissen/Datenänderungen). Jetpack Compose nutzt zudem Kotlin, was die Einbindung in Android stark erleichtert.⁴⁵

³⁸ Vgl. V.N. Lukin A. L. Dzyubenko, Y. B. C. (2019), 317f.

³⁹ Vgl. Chlebek, P. (2011), 29ff.

⁴⁰ Vgl. ebenda, S. 44.

⁴¹ Vgl. ebenda, 66f.

⁴² Vgl. ebenda, 129ff.

⁴³ Vgl. Sommerville, I. (2016), 62ff.

⁴⁴ Vgl. Chlebek, P. (2011), 161ff.

⁴⁵ Vgl. Asefa, B. (2022), S. 2–8.

2.4 Zustandsdiagramme in der Entwicklung von grafischen Benutzeroberflächen

Wie in sonstigen Softwareentwicklung können auch in der Entwicklung von Benutzeroberflächen UML-Diagramme zum Zweck der Modellierung eingesetzt werden. Chlebek nennt unter anderem Klassendiagramme, Use-Case-Diagramme und Robustheitsdiagramme als Analyse- und Entwurfswerkzeuge.⁴⁶ In einem Nebensatz wird die Nutzung von Statecharts zur Modellierung von Interaktionen erwähnt (wobei Rumpe⁴⁷ widersprechen würde, laut ihm sollten Zustandsdiagramme nur zur Modellierung Objektverhalten genutzt werden).

Kistner und Nuernberger von NVIDIA⁴⁸ haben die Anwendung „Architect“ entwickelt. Die Anwendung basiert auf „NVIDIA’s UI Composer Studio“, einer Anwendung zur Erstellung von 3D-Benutzeroberflächen. Das Ziel der Anwendung war, visuelle Zustandsänderungen von ihren Auslösern, um den Entwicklungsprozess zu erleichtern. Für die Darstellung der logischen Zustände wurden Zustandsdiagramme genutzt. „Architect“ ist ein visueller Editor und erlaubt die Erstellung und Modifizierung von SCXML (Statechart-XML) Dateien. Mit Hilfe einer zwischengestellten Runtime wurde die Zustandssteuerung von Benutzeroberflächen mit SCXML-Zustandsdiagrammen realisiert.

Besonders relevant ist hier die Arbeit von Nguyen.⁴⁹ In deren Arbeit werden Zustandsdiagramme genutzt um das Verhalten der Musikspielapplikation „XTune“ zu modellieren. Für die Programmierung der App wurde Vuex, ein progressives Webframework, genutzt, das in seiner Funktionsweise Jetpack Compose ähnelt. Zuerst wurden Funktionen der Anwendung als funktionale Anforderungen in Form von User-Stories definiert. Anschließend wurden für die einzelnen Funktionen jeweils ein Zustandsdiagramm modelliert. Die Modelle wurden auf Validität, Erreichbarkeit der Zustände, Determinismus und sog. „Deadlock“-Zustände (Zustände, die nicht terminieren und nicht gewollt sind) geprüft. Mit Hilfe von XState, einem Typescript-basierten SCXML-Interpreter, wurden die Zustandsdiagramme in der Applikation umgesetzt.

⁴⁶ Vgl. Chlebek, P. (2011), S. 44.

⁴⁷ Vgl. Rumpe, B. (2011), S. 149.

⁴⁸ Vgl. Kistner, G./Nuernberger, C. (2014).

⁴⁹ Vgl. Nguyen, T. (2020).

3 Methodik

3.1 Forschungsmethode

Diese Arbeit ist die Prüfungsleistung im Modul „Softwareprojekt“ an der Dualen Hochschule Sachsen in Dresden. Die Auswahl der Fallstudie als Forschungsmethode liegt in der Vorgabe durch den Dozenten begründet.

Fallstudien ordnen sich in die empirische qualitative Forschung ein.⁵⁰ Das Subjekt der Fallstudie ist ein aktuelles, soziales Phänomen mit seinem Kontext in der realen Welt; der Fall.⁵¹⁵² Jene Fälle können unter anderem Individuen, kleine Gruppen, Organisationen, Projekte, Gemeinschaften, Beziehungen, Entscheidungen oder Partnerschaften sein.⁵³ Die Daten können aus verschiedenen Quellen wie Dokumenten, Interviews, Observation(sprotokolle) oder physischen Artefakten erhoben werden.⁵⁴⁵⁵ Fallstudien können anhand der Anzahl von betrachteten Fällen in Einzelfallstudien (ein Fall) und vergleichende Fallstudien (mehrere Fälle) unterschieden werden.⁵⁶ Riedl unterscheidet zudem auf Basis des Zwecks in Deskriptive Fallstudien (Beschreibung von Artefakten, Sachverhalten, Ereignissen und Handlungen), Explorative Fallstudien (Generierung von Hypothesen) und Explanative Fallstudien (Testung von Theorien).⁵⁷

3.2 Vorgehen

Diese Fallstudie ist als deskriptive Fallstudie konzipiert. Zudem wird in dieser Fallstudie ein Einzelfall betrachtet. Beide Entscheidungen werden durch den limitierten zeitlichen Rahmen begründet.

In dem folgenden Kapitel erfolgt eine Beschreibung des Falls. Anschließend sollen Daten aus dem Fall analysiert werden. Als Datenquellen sollen folgende Dokumente und digitale Artefakte betrachtet werden.

- Zustandsdiagramme
- Software Requirements Specification
- Quellcode der Software

⁵⁰ Vgl. Ridder, H.-G. (2020), S. 113.

⁵¹ Vgl. ebenda, S. 113.

⁵² Vgl. Riedel, M. D. R. (2006), S. 117.

⁵³ Yin, R. K. (2018), S. 71.

⁵⁴ Vgl. Ridder, H.-G. (2020), S. 86.

⁵⁵ Vgl. Heinzl, A./Mädche, A./Riedl, R. (2024), S. 65.

⁵⁶ Vgl. Ridder, H.-G. (2020), 117ff.

⁵⁷ Vgl. Riedel, M. D. R. (2006), 130f.

- Bildschirmfotos der Benutzeroberfläche

In der Datenanalyse soll der Zweck der Nutzung der Zustandsdiagramme identifiziert werden. Mithilfe der Software Requirements Specification, dem Quellcode der Software und den in Kapitel 2 beschriebenen theoretischen Grundlagen sollen die Zustandsdiagramme validiert und verifiziert werden. Der Zweck der Datenanalyse ist die Beantwortung der in Kapitel 1 gestellten Forschungsfrage.

Es ist zu erwähnen, dass der Autor dieser Arbeit in dem Fall unfänglich als Akteur beteiligt war. Trotzdem soll in dieser Arbeit ein neutral beobachtender Standpunkt angenommen werden.

4 Empirische Beobachtung

4.1 Fallbeschreibung

4.1.1 Das Gartenbahnprojekt

Das Modul „Softwareprojekt“ an der Dualen Hochschule Sachsen in Dresden wird anhand einer „Gartenbahn“ durchgeführt. Die Gartenbahn konstituiert mehrere hölzerne Kästen, auf denen unter anderem Miniaturmodelle von Kränen, Bahnhöfen und Landschaftselementen, und stromleitende Miniaturschienen der Spurweite G angebracht sind. Jene stromleitende Schienen können Miniaturlokomotiven antreiben. Die Lokomotiven und diverse andere Miniaturmodelle der Gartenbahn (Weichen, Kräne, Schranken) sind mit Raspberry Pi's ausgestattet (Modellserver), welche eine kabellose Steuerung der ermöglichen. An der Unterseite der Module sind ebenfalls Raspberry Pi's angebracht, die als Namensserver für die Modellserver fungieren.

Wenn die Gartenbahn an Strom angeschlossen wird, versuchen sich die einzelnen Modellserver mittels HTTP-Anfragen bei einem Namensserver „anzumelden“. Ist die Anmeldung erfolgreich, können die jeweiligen Modellserver mittels einer HTTP-GET-Anfrage an den Namensserver ermittelt werden. Um die einzelnen Modellserver anzusprechen, wurde von dem Dozenten der Vorlesung ein Client für Android-Geräte in Kotlin entwickelt. Der Client führt die oben beschriebene HTTP-Get-Anfrage durch und zeigt die zurückgegebenen Server als Elemente der Gartenbahn an. Wird ein bestimmtes Modell ausgewählt, baut der Client eine Websocket-Verbindung mit dem korrespondierenden Modellserver auf. Der Client stellt die verschiedenen Attribute des Modellobjekts mittels seiner grafischen Benutzeroberfläche dar. Die Benutzeroberfläche des Clients ist in Abbildung 6 dargestellt. Über die Benutzeroberfläche können die Attribute des Modells wie beispielsweise die Geschwindigkeit einer Lokomotive geändert werden. Durch dieses Prinzip können die Elemente der Gartenbahn gesteuert werden.

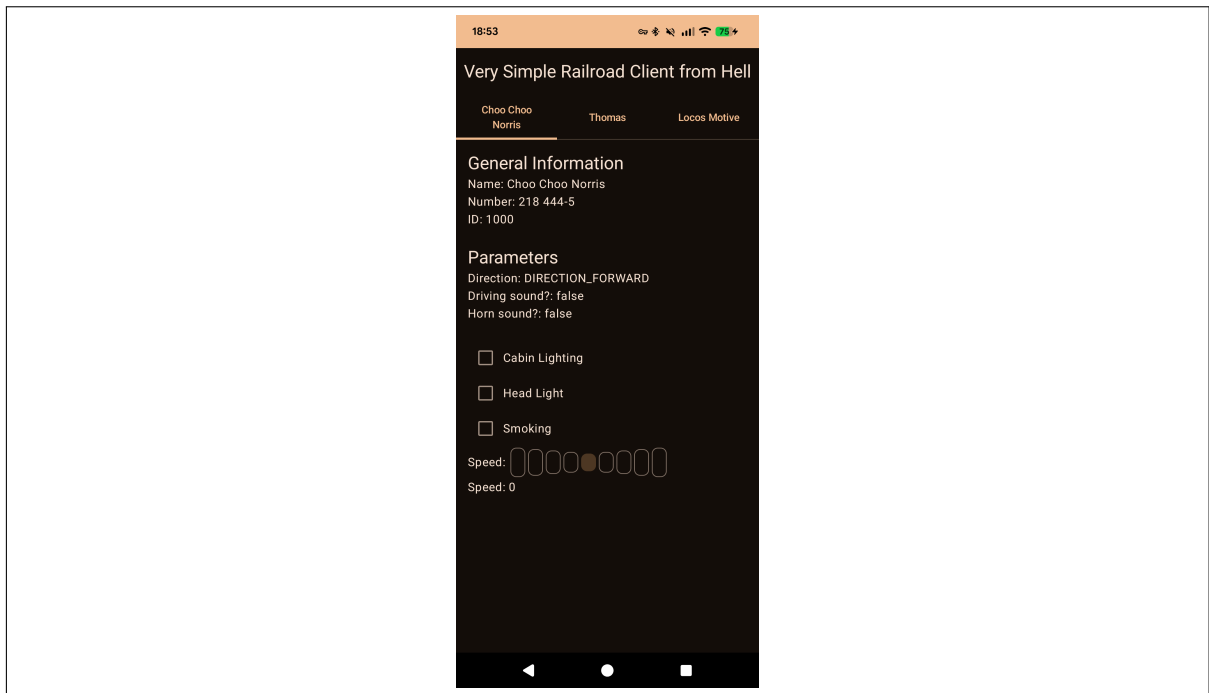


Abbildung 6: Benutzeroberfläche des Clients. Die obere Tableiste stellt die Liste an verfügbaren Modellen dar (Quelle: Eigene Darstellung)

4.1.2 Der Controlleur

Im Rahmen des Softwareprojekts entschied sich die Gruppe 2 mit den Mitgliedern Jessica Driver, Pia Ihbe, Miriam Herzog, Benjamin Lehmann, Luca Schubert und Christoph Höntzsch für die Entwicklung einer Viedospielcontrollersteuerung der Gartenbahn, spezifisch die Steuerung von Lokomotiven/Bahnen. Das resultierende Produkt trägt den Namen „Der Controlleur“ und wurde von dem fiktiven Unternehmen „ControlWare“ entwickelt.

Die Anforderungserhebung wurde durch eine Online-Umfrage durchgeführt, welche durch die Teammitglieder Benjamin Lehmann, Miriam Herzog und Jessica Driver konzipiert wurde. Anhand der Ergebnisse wurde eine Persona erstellt und Anforderungen wurden in die Software Requirements Specification übernommen. Im Anschluss wurde die Softwarearchitektur durch Pia Ihbe festgelegt. Aus zeitlichen und Komplexitätsgründen wurde entschieden, die bestehende Client App mit der Controllersteuerung zu erweitern. Die Entwicklung der funktionalen Einbindung des Controllers fand durch Luca Schubert statt. Für ein Redesign der Benutzeroberfläche war Christoph Höntzsch verantwortlich. Für die Entwicklung der Benutzeroberfläche wurde GitHub Copilot mit GPT4.1 als Basismodell unterstützend genutzt.

4.1.3 Redesign der Benutzeroberfläche

Aufgrund des limitierten zeitlichen Rahmens wurde sich für die Wasserfallmethode beim Redesign der Benutzeroberfläche entschieden. Wie oben beschrieben wurde die Anforderungserhebung durch eine Online-Umfrage ausgeführt. Aus den in der Software Requirements Specification ausformulierten Anforderungen war für den Entwickler nur NFA-03 relevant.⁵⁸ Dadurch, dass „die Steuerung der Gartenbahn [...] dem Nutzer intuitiv erscheinen und visuell nahegebracht werden [muss]“, wurde entschieden, dass die Steuerungselemente des Controllers (Knöpfe, Tasten) auf der Benutzeroberfläche mit den korrespondierenden Steuerungsfunktionen abgebildet werden sollten. Dadurch, dass „das System [...] wichtige Zustandsänderungen mit visuellen oder haptischen Feedback bestätigen [sollte]“, wurde entschieden, dass Elemente der Benutzeroberfläche die Zustände der Lokomotiveneigenschaften und die Zustände der Steuerungselemente des Controllers darstellen dynamisch sollten. Auf Basis dieser Entscheidungen wurde ein Mockup mit der browserbasierten Designsoftware Figma erstellt. Das Mockup ist in Abbildung 7 dargestellt. Basierend auf dem Mockup wurden zudem drei Zustandsdiagramme erstellt, die den Zustand der verschiedenen Arten von Elementen der Benutzeroberfläche modellieren. Die Zustandsdiagramme sind in den Abbildungen 8, 9 und 10 dargestellt.

⁵⁸ Vgl. Driver, J. et al. (2025), S. 11.

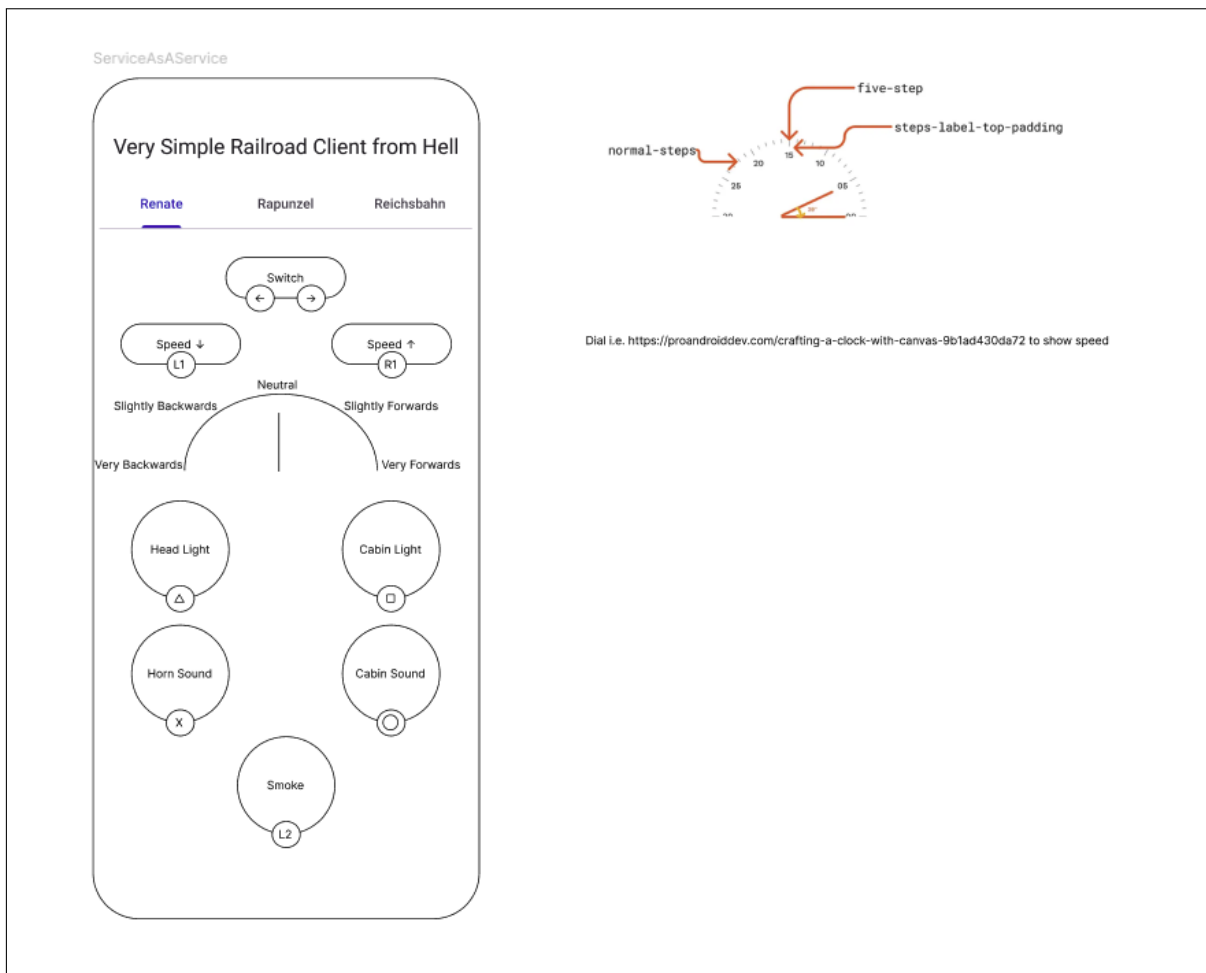


Abbildung 7: Mockup der Benutzeroberfläche (Quelle: Eigene Darstellung)

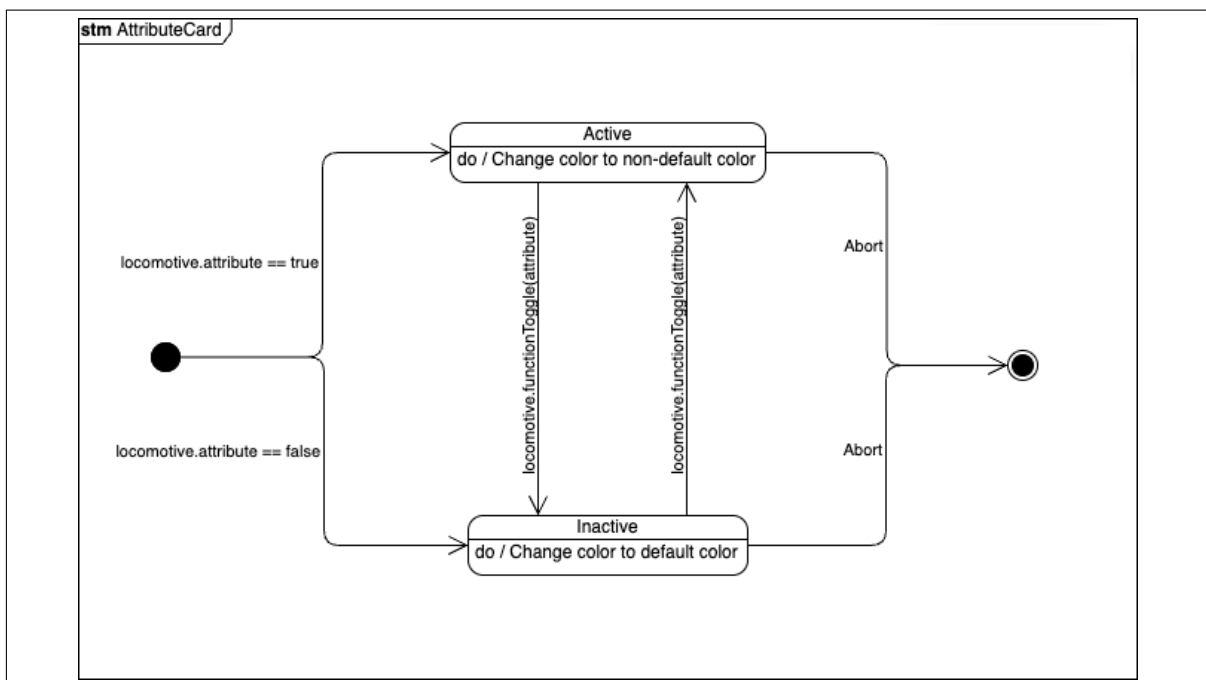


Abbildung 8: Zustandsdiagramm AttributeCard (Quelle: Eigene Darstellung)

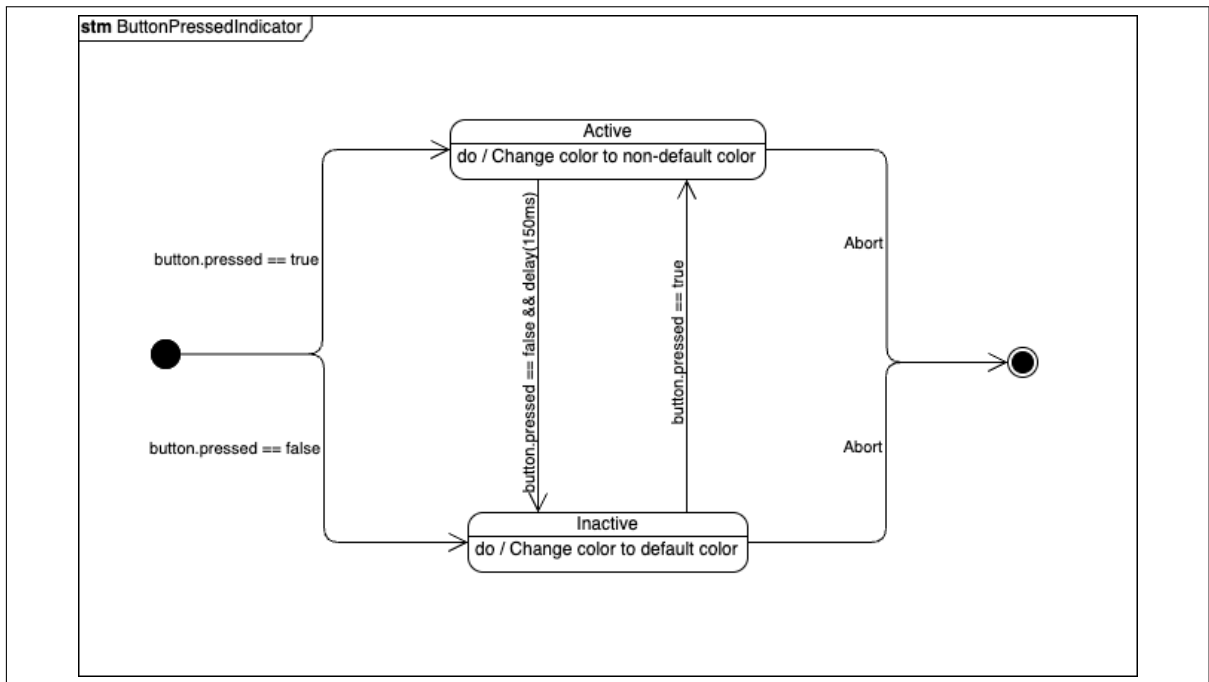


Abbildung 9: Zustandsdiagramm ButtonPressedIndicator (Quelle: Eigene Darstellung)

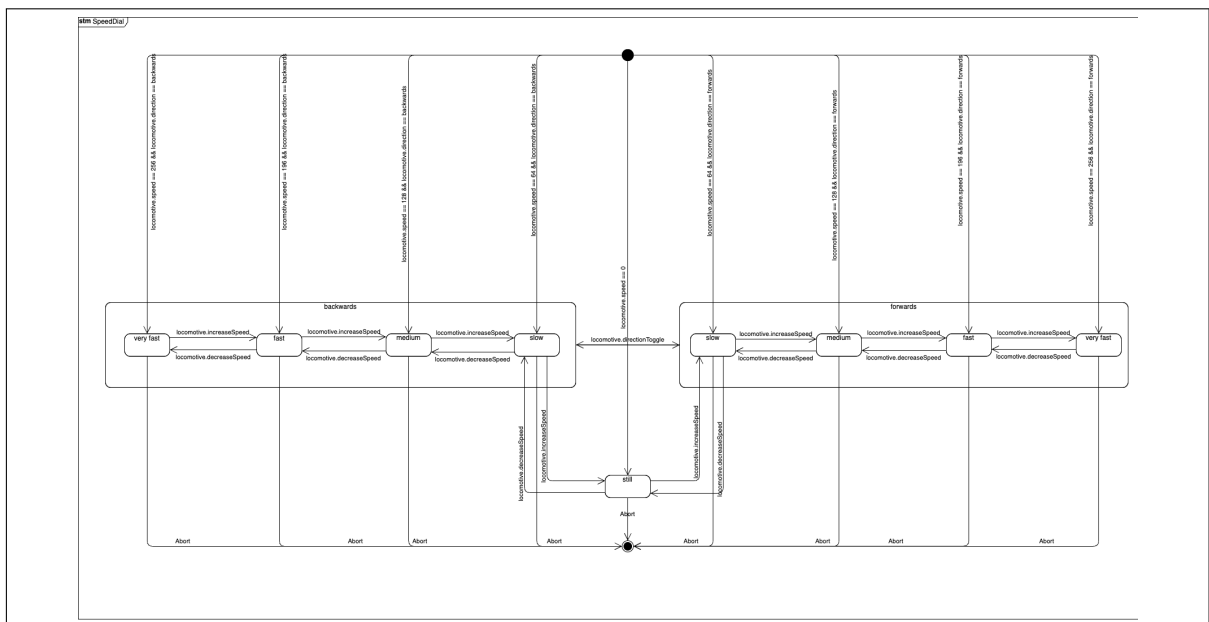


Abbildung 10: Zustandsdiagramm SpeedDial (Quelle: Eigene Darstellung)

Die für die Implementierung relevanten Dateien sind HardwareController.kt, LocomotiveController.kt, RailroadViewModel.kt, NewChips.kt, SpeedDial.kt und LocomotiveScreen.kt.

In LocomotiveController.kt definiert die Klasse ControllerInput die Funktionen, die durch eine Controllereingabe ausgeführt werden können. Eine Klasse FunctionType wurde hinzugefügt, um zwischen den verschiedenen Attributen einer Lokomotive zu unterscheiden, die ein-/ausgeschaltet werden können.

In HardwareController.kt werden in der Klasse HardwareController die verschiedenen

KeyEvents, die erzeugt werden, wenn ein Knopf oder eine Taste gedrückt wird, den verschiedenen Funktionen zugeordnet. Hier erfolgte eine Erweiterung um eine Funktion `processMotionEvent`, um die Eingabe von non-binären Schultertasten zu ermöglichen.

In `RailroadViewModel.kt` wurde in der Klasse `RailroadViewModel` die Datenklasse `ControllerHighlightState`, die die booleschen Zustände der hervorgehobenen Benutzeroberflächenelemente als Attribute enthält, definiert und als `MutableStateFlow` instanziiert. Der `MutableStateFlow` erlaubt das Teilen der im Flow enthaltenen Daten zwischen mehreren Objekten und der Aktualisierung der Objekte nach einer Änderung der Daten. Zudem wurde eine Funktion `pulseHighlight` hinzugefügt, die den `MutableStateFlow` mit neuen Daten aktualisiert, 150 Millisekunden wartet und dann den `MutableStateFlow` zurücksetzt. Durch die Attribute im `MutableFlow` werden die Elemente der Benutzeroberfläche konditionell gefärbt. In der Funktion `handleControllerInput` wurde für jede in `ControllerInput` definierte Funktion die Funktion `pulseHighlight` hinzugefügt, wobei für jede definierte Funktion das korrespondierende Attribut im `MutableStateFlow` auf `true` geändert wird.

Die tatsächliche Benutzeroberfläche beschränkt sich auf die Dateien `LocomotiveScreen.kt`, `NewChips.kt` und `SpeedDial.kt`. In `NewChips.kt` werden die Benutzeroberflächenelemente definiert, die die Funktionen des Controllers, die Tastenbelegung und teilweise den Zustand der Attribute der Lokomotive anzeigen. Für die Darstellung der Geschwindigkeit und Richtung wurde aus kreativen Zwecken ein Tachometer gewählt, das in `SpeedDial.kt` definiert ist. `LocomotiveScreen.kt` enthielt die originale Benutzeroberfläche für die Steuerung der Lokomotiven. Die neuen Elemente aus `SpeedDial.kt` und `NewChips.kt` werden in `LocomotiveContent` zu einer kohärenten Benutzeroberfläche zusammengestellt. Zudem werden die Elemente mit dem `MutableStateFlow` und dem geladenen Lokomotivenobjekt verbunden. Die finale Version der Benutzeroberfläche ist in den Abbildungen 11, 12 und 13 dargestellt.



Abbildung 11: Die entstandene Benutzeroberfläche (Quelle: Eigene Darstellung)



Abbildung 12: Die Benutzeroberfläche ändert sich durch das Drücken eines Knopfes (Quelle: Eigene Darstellung)

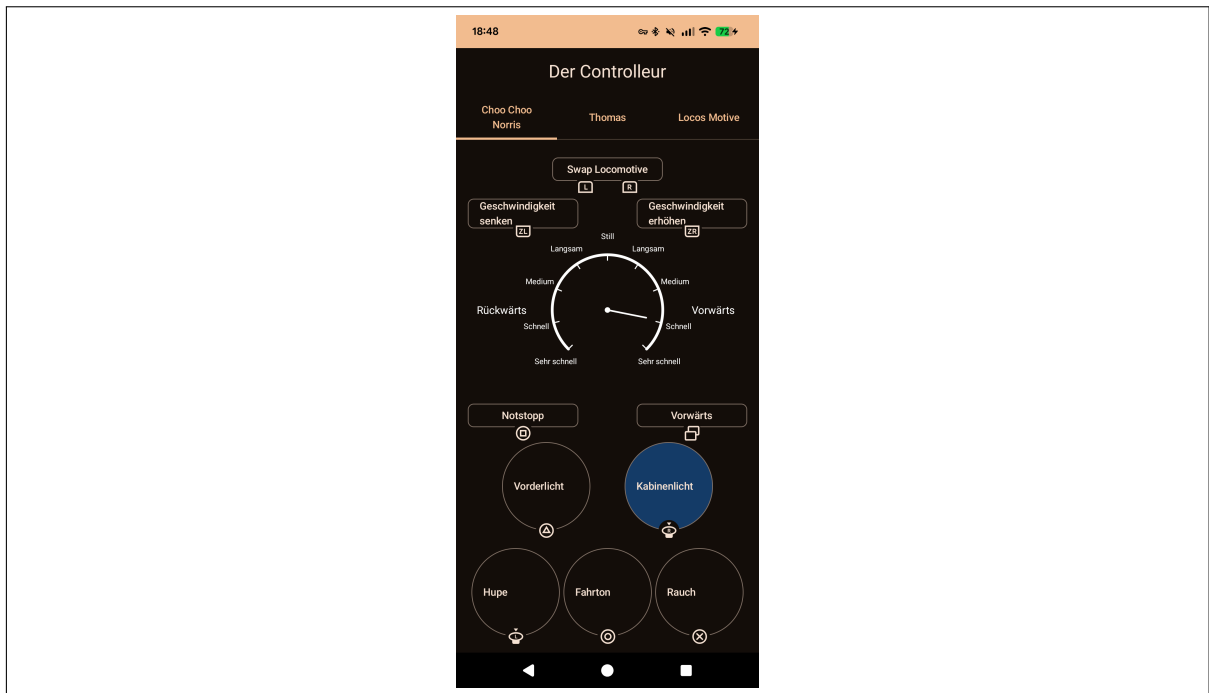


Abbildung 13: Die Benutzeroberfläche ändert sich, weil die Geschwindigkeit der Lokomotive geändert und ein Attribut der Lokomotive auf true gesetzt wurde (Quelle: Eigene Darstellung)

5 Analyse

In diesem Abschnitt sollen die in Kapitel 1 formulierten Analyseziele durch die Beantwortung von Fragen erreicht werden.

Wie wurden Zustandsdiagramme verwendet?

In Anbetracht der von Rumpe genannten Anwendungszwecke (vgl. Abbildung 5) können die Zustandsdiagramme in die Darstellung des Lebenszyklus bzw. Implementierungsbeschreibung des Verhaltens eines Objekts eingeordnet werden. Die Objekte sind in diesem Fall Elemente der Benutzeroberfläche. Das Diagramm `ButtonPressedIndicator` modelliert die unten an den Chips sich befindenden Icons, die bei Drücken des korrespondierenden Knopfes die Farbe ändern, das Diagramm `AttributeCard` modelliert die Chips, die ein Attribut der Lokomotive widerspiegeln, und das Diagramm `SpeedDial` modelliert das Tachometer. Die verschiedenen Anzeigeformen der Elemente der Benutzeroberfläche, die aufgrund von Änderungen in den zugrundeliegenden Daten entstehen, werden als Zustände aufgefasst. Die Schaltbedingungen werden teilweise als Methode eines Objekts, teilweise als Boolean dargestellt. Das kann daran liegen, dass manche Methodenaufrufe als Trigger für eine Zustandsänderung dienen, dass im Code die Zustände teilweise auch als konditionelle oder booleansche Variablen dargestellt werden und eine Änderung der Kondition eine Änderung des Zustands darstellt. Die Zustandsdiagramme wurden verwendet, um die Zustandslandschaft der einzelnen Benutzeroberflächenelemente zu modellieren. Zusätzlich konnten relevante Variablen und Methoden identifiziert und modelliert werden, die die Zustandsänderung hervorrufen. Unter der Prämisse, dass nur Menschen einen Knopf betätigen sollen, kann das Diagramm `ButtonPressedIndicator` auch als Modell einer Interaktion, spezifisch der Mensch-Maschine-Interaktion, interpretiert werden. Dies widerspricht jedoch der Prämisse aus Rumpe⁵⁹, dass Zustandsdiagramme nur für die Verhaltensmodellierung und nicht für die Modellierung von Interaktionen genutzt werden sollen.

Sind die Zustandsdiagramme korrekt?

Die Diagramme sind syntaktisch korrekt. Es wurden folgende semantische Unklarheiten bzw. Fehler gefunden gefunden:

1. Die Notation für Transitionen ist $[\langle \text{trigger} \rangle [\text{'}, \text{' } \langle \text{trigger} \rangle]^* [\text{' } \langle \text{guard} \rangle \text{' }] [\text{' } / \text{' } \langle \text{behavior-expression} \rangle]]$.⁶⁰ Einen Trigger als einen Boolean zu notieren ist syntaktisch nicht falsch, kann aber zu Verwirrung führen, da Booleans meistens für guard-Bedingungen verwendet werden.

⁵⁹ Vgl. Rumpe, B. (2011), S. 149.

⁶⁰ Vgl. Group, O. M. (2017), S. 331.

2. Im Diagramm SpeedDial existieren zwei Transitionen zwischen den Zuständen „backwards“ und „forwards“. Da die Zustände Subzustände enthalten, jedoch keinen Pseudo-Startzustand, ist kein sog. Default Entry Behavior definiert. Daraus folgt, dass der Zustandsautomat nach der Transition terminiert oder das Diagramm formal ungültig ist.⁶¹

Konnte der Verwendungszweck erfüllt werden?

Angesichts der im vorherigen Paragraph benannten Fehler wäre eine Verneinung der Frage durchaus berechtigt. Es ist jedoch zu bemerken, dass neben einer Sicht des Modells auf das modellierte Objekt ebenfalls die Sicht des Betrachters auf das Modell existiert, was eine subjektive Sichtweise auf ein Modell bedingt. Dies ermöglicht die Bildung eines semantisch kohärenten Modells innerhalb der subjektiven Sichtweise entgegen des semantisch inkorrekten Modells, insbesondere wenn der Betrachter sich nicht den unterliegenden Formalismen zur Betrachtung des Modells bekannt ist. In Anbetracht dessen, dass die Benutzeroberfläche von genau einer Person implementiert wurde, kann der Verwendungszweck, das Modell als Vereinfachung eines Gegenstands zur Erleichterung der Untersuchung zu nutzen, durchaus erfüllt werden. Ob andere geplante Verwendungszwecke erfüllt werden konnten, kann an dieser Stelle nicht geprüft werden.

⁶¹ Vgl. Group, O. M. (2017), 309f.

6 Verifizierung und Validierung

6.1 Verifizierung als wissenschaftliches Schreibprojekt

Die Verifizierung dieser Arbeit als wissenschaftliche Arbeit orientiert sich an Abschnitt 1.3 „Allgemeine Anforderungen an wissenschaftliche Schreibprojekte“ des Leitfadens für wissenschaftliche Arbeiten (verk.)⁶². Es werden relevante Anforderungen als Zitate aus der Quelle übernommen und geprüft.

„Dafür verfügen Sie über die notwendigen Fachkenntnisse. Genaugenommen ist deren Erarbeitung also nicht Teil Ihrer Schreibprojekte.“

Die Fachkenntnisse, die für die Erstellung dieser Arbeit notwendig waren, waren vor der Erstellung nicht ausreichend vorhanden. Dies ist bedingt durch die Themenauswahl, die durch den von dem Dozenten gesetzten Rahmen ermöglicht wurde, eine spezielle Teilmenge der eigentlichen Themen darstellt (UML - Zustandsdiagramme, Benutzeroberflächen - Benutzeroberflächenentwicklung) und zusätzliche Literaturrecherche benötigte. Nach der Bearbeitung wurden die notwendigen Fachkenntnisse akquiriert.

„Um Ihren Eigenbeitrag tatsächlich beurteilen zu können, ist es wichtig, dass Sie sorgfältig kennzeichnen, welche Inhalte Sie von anderen übernommen haben“

Die Zitierung wird den formalen Vorgaben gerecht vorgenommen. Eine Dokumentation der KI-Nutzung wird in einem zusätzlichen Dokument vorgenommen. Die Gliederung dieser Arbeit wurde von Michael L. inspiriert. LM Sonstige Hilfe wurde nicht bezogen.

„Darüber hinaus haben Sie Ihre Arbeiten immer auf der Grundlage anerkannter wissenschaftlicher Prinzipien Ihrer jeweiligen Disziplin anzufertigen.“

Relevante Prinzipien (Erkenntnisorientierung, Nachvollziehbarkeit, Empirische Fundierung, Theoriebezug, ...) wurden beachtet.

„[...] Verwendung von anerkannten [, aufgabengerechten] Methoden Ihres Fachgebietes bzw. Ihrer Disziplin.“

Die Fallstudie ist eine in der Wirtschaftsinformatik anerkannte Methode.

„Zudem wird von Ihnen erwartet, dass Sie systematisch an die Bearbeitung resp. Lösung Ihrer jeweiligen Aufgaben- bzw. Problemstellung herangehen.“

⁶² Grassmann, D. T. (2017).

Das System für die Lösung der Aufgabe war durch die Struktur der Lehrveranstaltung sowie der Methodik vorgegeben.

Insgesamt ist eine Nichterfüllung der ersten Anforderung zu verzeichnen. Damit ist diese Arbeit kein wissenschaftliches Schreibprojekt.

6.2 Validierung

6.2.1 Wissenschaftliche Relevanz

Disterer definiert wissenschaftliche Aussagen als relevant, wenn sie „Fragen von Interesse beantworten“.⁶³ Die wissenschaftliche Aussage dieser Arbeit beantwortet die Forschungsfrage „Wie können UML-Zustandsdiagramme in der Entwicklung von Benutzeroberflächen genutzt werden?“. Das Interesse des Autors an der Frage ist durch die Aufforderung des Dozenten, Interesse an den bearbeiteten Themen und der Beantwortung der Frage zu haben, begründet. Für die breitere wissenschaftliche Gemeinschaft könnte die Frage von Interesse sein, da deklarative Oberflächen in der Benutzeroberflächenentwicklung starkes Wachstum verzeichnen. In der Webentwicklung sind Web-Technologien wie React, Next.js, Vue.js oder Svelte sowohl bei angehenden als auch bei professionellen Entwicklern sehr beliebt.⁶⁴ Die Bedeutung von Zustandsdiagrammen für zustandsbasierte Benutzeroberflächen konnte nicht vollständig geklärt werden. Diese Arbeit soll jedoch einen Ansatz für die Verwendung geben.

Die wissenschaftliche Aussage dieser Arbeit ist in ihrer Aussagekraft beschränkt. In der vorliegenden Fallstudie wurden nur Dokumente und Artefakte als Analyseobjekte betrachtet. Zudem wurde lediglich ein Fall betrachtet, der zudem kaum Bezug auf die tatsächliche Praxis der Benutzeroberflächenentwicklung aufgrund von unter anderem der Teamgröße hat. Die Konzeption als deskriptive Fallstudie bedingt zudem einen geringeren Erkenntniswert als eine vergleichbare explorative oder explanative Fallstudie.⁶⁵

6.2.2 Relevanz der Arbeit für den Autor

Diese Arbeit stellt für den Autoren die dritte (wissenschaftliche) Schreibarbeit an der Dualen Hochschule Sachsen in Dresden dar. Der Nutzen der Akquise der Fallstudie als wirtschaftswissenschaftliche Methodik ist voraussichtlich hoch, da sie in zukünftigen Arbeiten ebenfalls verwendet werden kann. Zudem ist ein hoher Wissenszuwachs in den Gebieten UML, Beschreibung von Standards, Finite Automatentheorie, Modellierung mit

⁶³ Vgl. Disterer, G. (2014), S. 23.

⁶⁴ Vgl. StackOverflow ().

⁶⁵ Vgl. Riedel, M. D. R. (2006).

Zustandsdiagrammen, Validierung und Verifizierung von Ergebnissen sowie der Gartenbahn zu verzeichnen.

7 Fazit und Ausblick

In dieser Arbeit sollte die Forschungsfrage „Wie können UML-Zustandsdiagramme in der Entwicklung von Benutzeroberflächen genutzt werden?“ beantwortet werden. Dazu wurden in Kapitel 2 die Grundlagen zu UML-Zustandsdiagrammen und der Entwicklung von Benutzeroberflächen. Für die Beantwortung der Frage wurde in Kapitel 3 die Fallstudie als Methodik gewählt. Der relevante Fall wurde in Kapitel 4 beschrieben und Dokumente und Artefakte wurden in Kapitel 5 analysiert. Anschließend wurde eine Verifizierung dieser Arbeit auf den Status als wissenschaftliche Schreibaarbeit und eine Validierung der Relevanz in Kapitel 6 durchgeführt.

In der Theoriebetrachtung wurden bereits andere Arbeiten identifiziert, die Zustandsdiagramme für die Entwicklung von Benutzeroberflächen genutzt haben. Es wurde identifiziert, dass Zustandsdiagramme zur Modellierung von Funktionen, und in Form von SCXML in der Entwicklung und Zustandsmodellierung von 3D-Oberflächen sowie der Implementierung durch XState genutzt werden können.

Anhand des beschriebenen Falls konnte ein weiterer potentieller Anwendungsfall identifiziert werden. UML-Zustandsdiagramme können in der Benutzeroberflächenentwicklung zur Modellierung der Zustandslandschaften von Elementen der Benutzeroberfläche genutzt werden. Zusätzlich können mit Hilfe der UML-Zustandsdiagramme für Zustandsänderungen relevante Variablen und Methoden identifiziert werden.

In zukünftigen Fallstudien kann der Rahmen der Fallstudie durch eine vergleichende oder explanative oder explorative Fallstudie erweitert werden. Zudem können praxisnähere Fälle aus beispielsweise Unternehmens- oder Open-Source-Kontexten genutzt werden, um unter anderem die Anwendbarkeit bei größeren Entwicklerteams zu bestimmen.

Literatur

- Aggarwal, M./Sabharwal, S. (2012): „Test Case Generation from UML State Machine Diagram: A Survey“, in: 2012 Third International Conference on Computer und Communication Technology, S. 133–140.
- Asefa, B. (2022): „Building Android Component Library Using Jetpack Compose“, Metropolia University of Applied Sciences.
- Chlebek, P. (2011): *Praxis der User Interface-Entwicklung. Informationsstrukturen, Designpatterns, Vorgehensmuster*, 1 Auflage, VIEWEG+TEUBNER, ISBN: 978-3-8348-0728-1.
- Disterer, G. (2014): *Studienarbeiten schreiben. Seminar-, Bachelor, Master- und Diplomarbeiten in den Wirtschaftswissenschaften*, 7 Auflage, Springer Gabler, ISBN: 978-3-642-55366-0.
- Driver, J. et al. (2025): *Software Requirements Specification*.
- Felderer, M./Herrmann, A. (2014): *Manual Test Case Derivation from UML Activity Diagrams and State Machines: a Controlled Experiment*, Gefunden über: Google Scholar.
- Grassmann, D. T. (2017): *Verbindlicher Leitfaden. für die Anfertigung und formale Gestaltung wissenschaftlicher (Haus-)Arbeiten an der Staatlichen Studienakademie Dresden*, in.
- Group, O. M. (2017): *OMG® Unified Modeling Language® (OMG UML®)*, Version 2.5.1, in.
- Guo, W. (/Zhou, M./Wei, D. (2007): „DiagramDraw: A State Machine Diagram Designer for Flexible Automation“, in: Proceedings of the 3rd Annual IEEE Conference on Automation Science und Engineering Scottsdale, AZ, S. 352–356.
- Harel, D. (1987): *Statecharts: A Visual Formalism for Complex Systems*, in: *Science of Computer Programming* 8 (1), S. 231–274.
- Heinzl, A./Mädche, A./Riedl, R. (2024): *Wirtschaftsinformatik. Einführung und Grundlegung*, 5 Auflage, Springer Gabler, ISBN: 978-3-662-67391-1.
- Jansen, B. J. (1998): *The Graphical User Interface. An Introduction*, in: *SIGCHI Bulletin* 30.2.
- Kecher, C./Hoffmann-Elbern, R./Will, T. T. (2021): *UML 2.5. Das umfassende Handbuch*, 7 Auflage, Leseprobe, Rheinwerk Computing, ISBN: 978-3-8362-8447-9.
- Kistner, G./Nuernberger, C. (2014): „Developing User Interfaces using SCXML Statecharts“, in: Gefunden über: Google Scholar, 41. Jahrestagung der Gesellschaft für Informatik.
- Martinez, W. L. (2011): *Graphical User Interfaces*, in: *WIREs Computational Statistics* 3 (2).
- Nguyen, T. (2020): „Statecharts for modern web application state management“, Metropolia University of Applied Sciences.
- Ridder, H.-G. (2020): *Case Study Research. Approaches, Methods, Contribution to Theory*, 2 Auflage, Rainer Hampp Verlag, ISBN: 978-3-95710-263-8.
- Riedel, M. D. R. (2006): „Erkenntnisfortschritt durch Forschungsfallstudien“, in: *Fortschritt in den Wirtschaftswissenschaften. Wissenschaftstheoretische Grundlagen und ex-*

emparische Anwendungen, 1 Auflage, Stephan Zelewski, Naciye Akca, S. 113–139, ISBN: 978-3-8350-0349-1.

Rumpe, B. (2011): *Modellierung mit UML*, 2 Auflage, Springer, ISBN: 978-3-642-22412-6.

Sommerville, I. (2016): *Software Engineering*, 10 Auflage, Pearson, ISBN: 978-1-292-09613-1.

StackOverflow (o. D.): *2025 Developer Survey. Technology*, Zuletzt aufgerufen am: 17.12.2025, in: <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-webframe>.

V., S. E./Samuel, P. (2019): *Automatic Code Generation From UML State Chart Diagrams*, in: *IEEEAccess* 7, S. 8591–8608.

V.N. Lukin A. L. Dzyubenko, Y. B. C. (2019): *Approaches to User Interface Development*, in: *Programming and Computer Software* 46.5, S. 316–323.

Yin, R. K. (2018): *Case Study Research and Applications: Design and Methods*, 6 Auflage, Sage, ISBN: 9781506336169.

Eidesstattliche Erklärung

Ich erkläre an Eides statt, dass ich die vorliegende Arbeit selbständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Stellen sind als solche kenntlich gemacht. Bestandteile der Arbeit, die mittels Künstlicher Intelligenz entstanden sind, wurden ausdrücklich gekennzeichnet. Die Zustimmung des/der beteiligten Unternehmen/s zur Verwendung betrieblicher Unterlagen habe ich eingeholt. Die Arbeit wurde bisher in gleicher oder ähnlicher Form weder veröffentlicht noch einer anderen Prüfungsbehörde/-stelle vorgelegt.

Hoentzsch, Christoph

Name, Vorname Verfasser

Dresden, 17.12.2025

Ort, Datum Abgabetermin



Unterschrift Verfasser